

Python程序设计基础

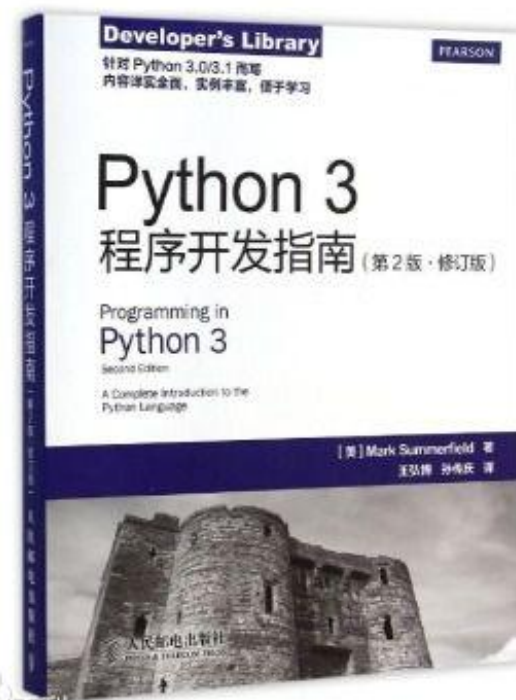
一、基础知识

教材

- 教材：Python程序设计基础，董付国，清华大学出版社，2015年8月
ISBN 978-7-302-41058-4



- 参考教材：Python 3程序开发指南，作者：Mark Summerfield[美]，人民邮电出版社，2011年 ISBN: 978-7-115-24507-6



计算机编程概述

- 程序(设计)语言是什么？
 - 程序：程序是用于控制计算机的一系列指令
 - 程序语言：描述这些指令的语言，是“程序员”与“机器”对话的语言。
 - 语法 (syntax)：哪些符号或文字的组合方式是正确的
 - 语义 (semantics)：描述程序的意义，当代码运行时计算机干什么。



程序语言



计算机编程概述

- 程序(设计)语言的发展

前辈



低级语言

例如：0101110，汇编

高级码农



高级语言

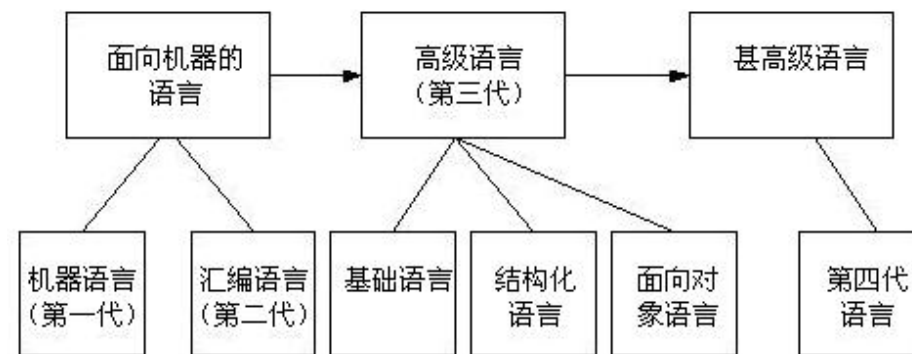
例如：C、C++、Java、Python

任何人



智能语言

例如：领域语言、自然语言



机器人：Atlas

计算机编程概述

- 程序(设计)语言的分类——解释型与编译型

- 解释器 (Interpreter)：将语句翻译成机器码，并且执行
 - 修改程序方便，修改代码重新运行就可以了
 - 每次运行，都要进行翻译，运行速度会有影响
 - 必须有解释器才可以运行、跨平台
 - Python、Javascript、Perl、PHP等
- 编译器 (Compiler)：将语句翻译成机器码，形成目标代码文件
 - 编译时相比解释可以作更多的优化
 - 修改程序后需要进行编译
 - 编译一次，然后在执行过程中不再需要翻译语句
 - 编译后的目标代码可以直接在相应的操作系统中运行，不再需要编译器
 - C、C++等
- 有些语言将解释和编译结合在一起
 - Java语言：源代码首先通过编译器 (javac) 转换为中间的Java字节码 (Byte Code)，然后在目标机器上通过解释器来运行
 - Python也支持伪编译，即将程序转换为字节码来优化程序和提高执行速度

- 说明：

- Python支持伪编译，将源程序转换为字节码来优化程序和提高运行速度
- 使用pyzexe工具将Python程序转换为扩展名为“.exe”的可执行程序，可以在没有安装Python解释器和相关依赖包的平台上运行

Python语言



- Python ['paɪθən]语言是一种解释型高级动态编程语言。
- 1989年，吉多·范罗苏姆Guido van Rossum发明Python。
- Python语言的前身是ABC语言。ABC是由Guido参加设计的一种教学语言。ABC语言是专门为非专业程序员设计的。
- Python的创始人为Guido van Rossum。1989年圣诞节期间，在阿姆斯特丹，Guido为了打发圣诞节的无趣，决心开发一个新的脚本解释程序，做为ABC语言的一种继承。



Python语言

- Python以接近自然语言的风格诠释程序设计，最受欢迎的编程语言之一。
- 广泛应用在系统管理、科学计算、大数据、Web应用、图形用户界面开发、游戏等
 - 引力波数据分析采用Python实现 *gwpv*: <https://github.com/gwpv/gwpv>



「学了Python才知道编程不止有快感，更有美感」

Google、Instagram、豆瓣、知乎，这些崇尚优雅的互联网公司都在用它

「人生苦短，快学Python」

小白迅速入门编程，Python是最棒的选择。

Python是世界上最容易学的编程语言，从没接触过编程的人也能搞定。

Python语言

- Python语言的特点*：
 - **简单、易学**：Python是一种代表简单主义思想的语言。
 - **开源、免费**：Python是FLOSS（自由/开放源码软件）之一。使用者可以自由地发布这个软件的拷贝、阅读它的源代码、对它做改动、把它的一部分用于新的自由软件中。
 - **跨平台**：所编写程序在解释器支持下可无需修改在Windows、Linux、Mac等操作系统上使用
 - **灵活性**：Python支持多种编程范式，包括面向对象编程、命令式编程、函数式编程，
 - **可嵌入性**：支持采用C、C++等语言编写扩充模块，也可为C、C++程序提供脚本功能
 - **丰富的扩展库支持**：拥有大量的几乎支持所有领域应用开发的成熟扩展库。

Python语言

- Python与其他语言的比较*:

简单 + 强大

	C++		JavaScript
	Java/C#		PHP (Without MySQL)
	Ruby		Pascal
	Perl		Lisp
	Visual Basic		Haskell
	Python		C

* <http://www.dataguru.cn/thread-190925-1-1.html>

Python之禅 (The Zen of Python)

*Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one -- and preferably only one -- obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right now*.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.*

— Tim Peters

Namespaces are one honking great idea -- let's do more of

- Python的设计哲学:



- Python开发者的哲学:

“用一种方法，最好是只有一种方法来做一件事”。

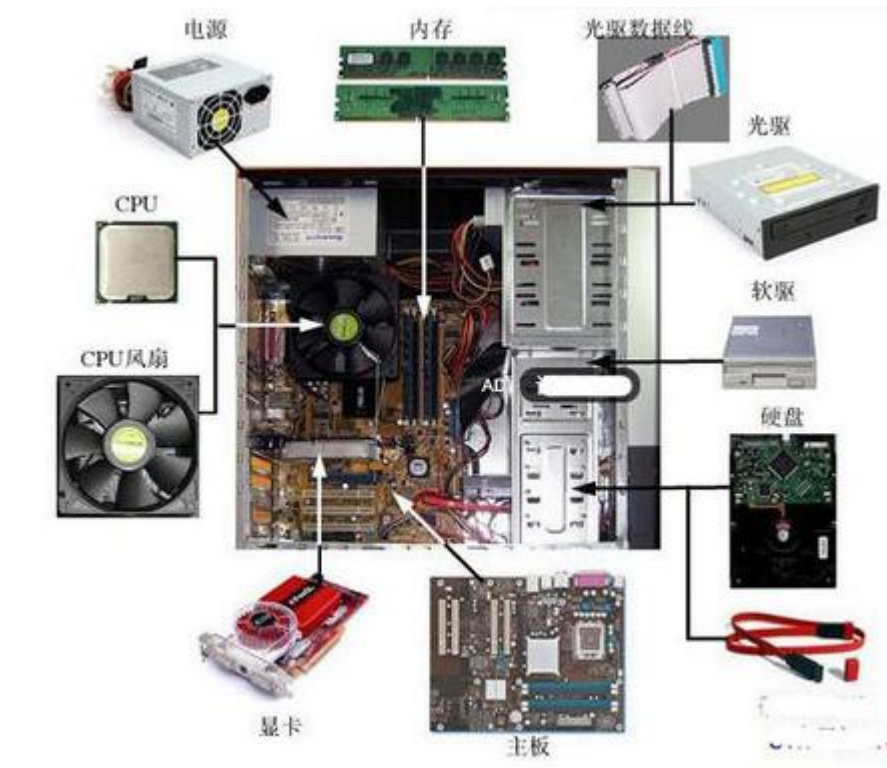
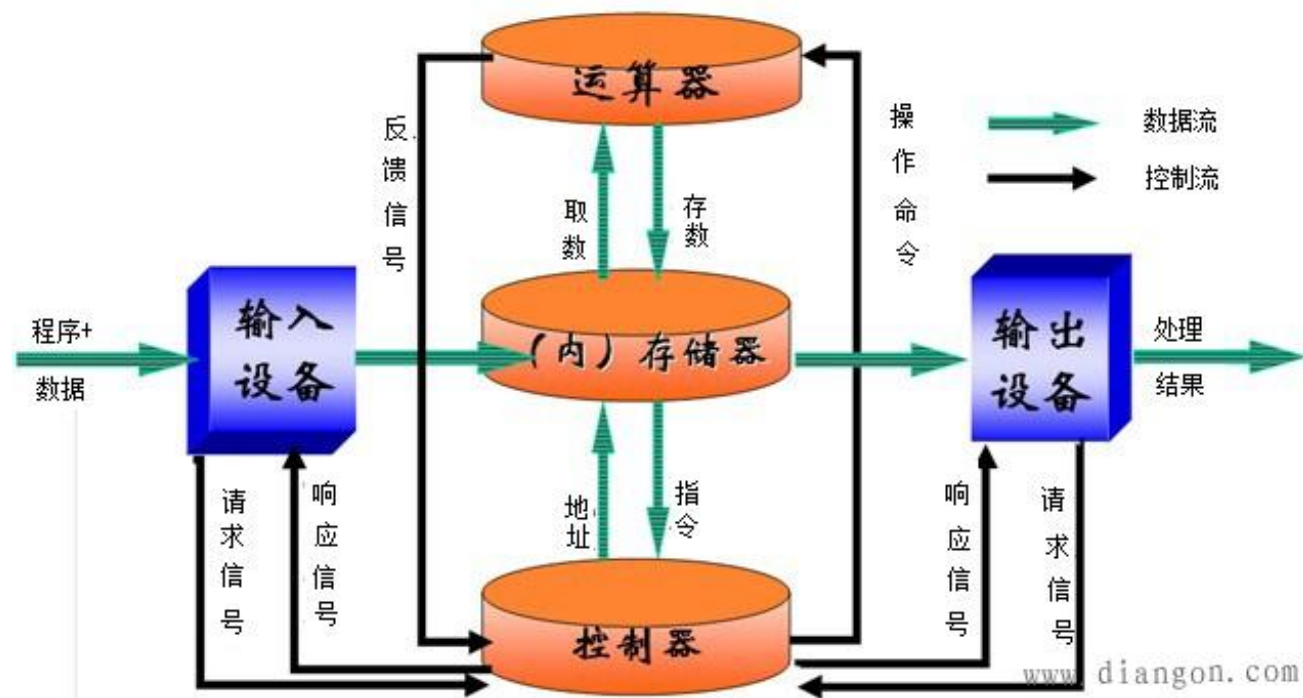
在设计Python语言时，如果面临多种选择，Python开发者总会拒绝花哨的语法，而选择明确的没有或者很少有歧义的语法。



pythonTM

1.0 预备知识

- 现代计算机的部件



数据表达： Bit、Byte(K/M/G)

- 计算机内部采用二进制表示

- 一个字节=8个比特 (1B=8b)
- 一个字节可以表示0到255的整数
 - 或者也可表示一个ASCII字符
- 二进制00101011对应十进制43

$$0 * 2^7 + 0 * 2^6 + 1 * 2^5 + 0 * 2^4 + 1 * 2^3 + 0 * 2^2 + 1 * 2^1 + 1 * 2^0 = 43$$

- Kilobyte (kB): $2^{10} = 1024$ bytes
- Megabyte (MB): $2^{20} = 1024$ kB or $1024 * 1024$ bytes
- Gigabyte (GB): $2^{30} = 1024$ MB
- Terabyte (TB): $2^{40} = 1024$ GB

基本编程套路：四部曲

- 一想 (Thinking)
- 二画 (Designing)
- 三码 (Coding)
- 四测 (Testing)

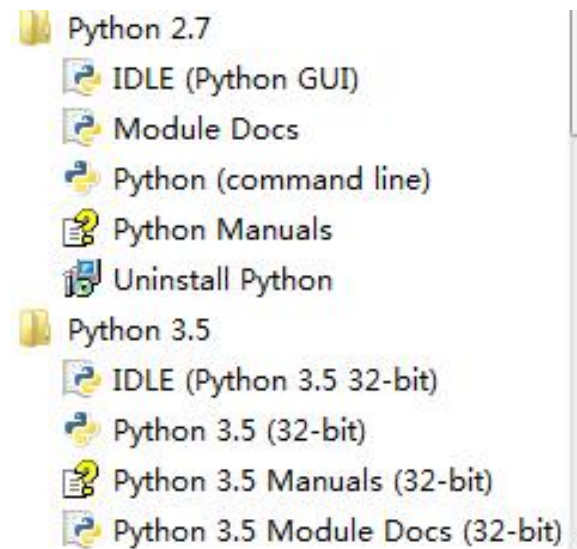
1.1 Python版本

- Python2:
 - 2000年10月发布，2.7在2010年发布，最新版本为2.7.11
 - 存在大量使用Python2开发的扩展库
- Python3
 - 2008年12月发布。2016年1月的最新版本为3.5.1
 - 3.x版本与2.x版本并不完全兼容，
 - Python3: `print('abc')`
 - Python2: `print 'abc'`
 - 绝大部分扩展库同时支持Python2和Python3，但是仍有少量扩展库仅仅支持Python2
 - Python提供2to3的转换工具，将支持Python2.7的代码转换为Python3
- 我们的教学采用Python 3，它是Python语言的现在和未来。
- 可同时安装多个版本，可通过更改系统环境变量path来决定缺省使用哪个版本
- 查看已安装版本的方法（在所启动的IDLE界面也可以直接看到）

```
import sys
sys.version
sys.version_info
```

1.2 Python安装

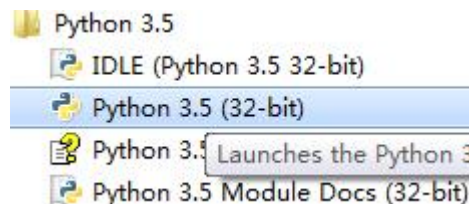
- Python解释器：
 - <https://www.python.org/downloads/> 根据所使用的平台（Linux/Mac OS X/Windows）下载相应的版本（3.5.1）
 - Install Now: 按照缺省设置安装
 - Customize installation: 可以修改安装目录等设置
 - Python Launcher for windows(py): 帮助选择合适的Python版本



Python使用:菜单选择Python控制台

- 通过菜单选择Python控制台(Console)

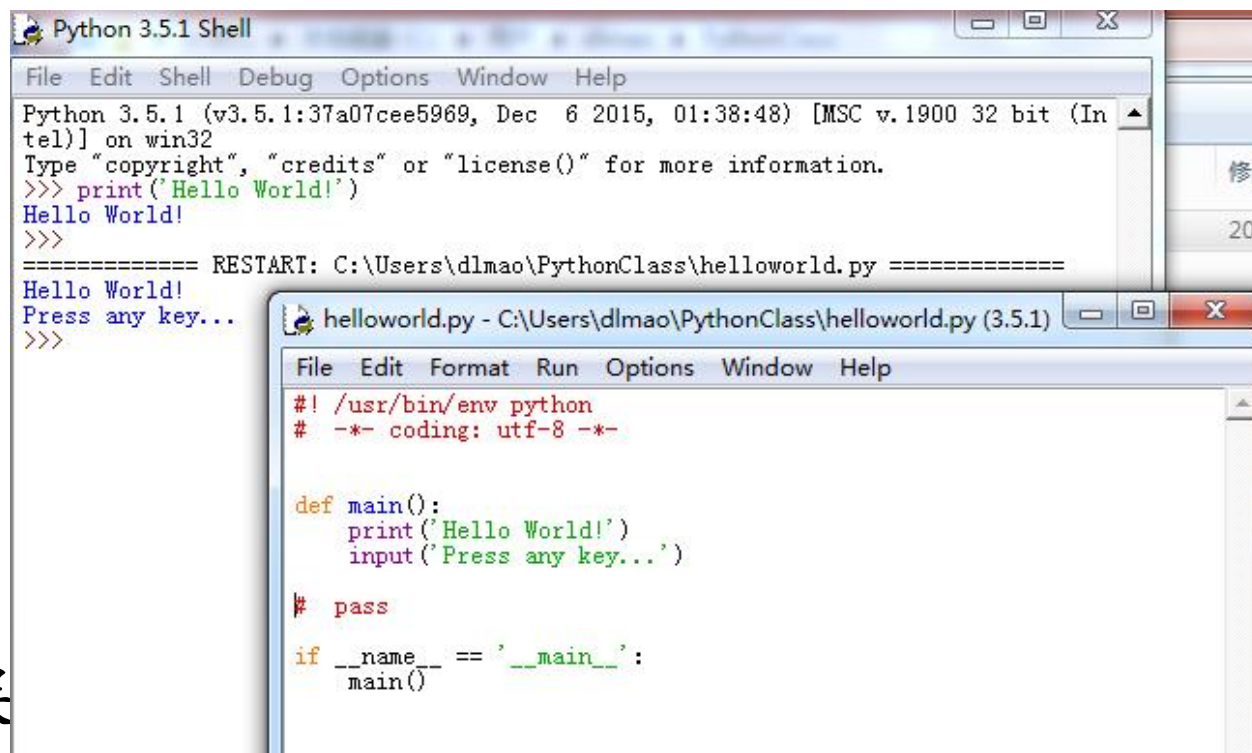
- Python解释器的提示符为 `>>>`
- 可输入语句，解释执行并输出结果
- 可以充当计算器的功能
- 解释器环境中 `_` 表示上一次运算的结果（注：仅在运行环境中起作用）

A screenshot of the 'Python 3.5 (32-bit)' console window. The window title is 'Python 3.5 (32-bit)'. The console text shows the Python version and environment information, followed by several commands and their outputs:

```
Python 3.5.1 (v3.5.1:37a07cee5969, Dec 6 2015, 01:38:48) [MSC v.1900 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import sys
>>> sys.version
'3.5.1 (v3.5.1:37a07cee5969, Dec 6 2015, 01:38:48) [MSC v.1900 32 bit (Intel)]'
>>> print('hello world!')
hello world!
>>> 15+4*8+9
56
>>> _//8
7
>>> _
```

Python使用:集成开发环境IDE (Integrated Development Environment)

- 将编辑器、调试器、解释器环境综合在一起
- 下载安装的Python环境中包括了IDLE
- 其他常用的IDE还包括PyCharm、WingIDE和Sublime
- File→New File: .py(或pyw)
- Run→Check Module检查语法错误
- Run→Run Module来运行程序



The image shows two overlapping windows from a Python 3.5.1 IDE. The background window is titled 'Python 3.5.1 Shell' and displays the Python interpreter's startup message, including version information and a prompt to type 'copyright', 'credits', or 'license()'. It shows a successful execution of `print('Hello World!')` which outputs 'Hello World!'. Below this, it indicates a restart of a script named `helloworld.py`. The foreground window is titled `helloworld.py - C:\Users\dlmao\PythonClass\helloworld.py (3.5.1)` and shows the source code of the script. The code includes a shebang line, a UTF-8 encoding declaration, a `main()` function that prints 'Hello World!' and prompts for input, and a standard `if __name__ == '__main__':` guard.

```
Python 3.5.1 (v3.5.1:37a07cee5969, Dec 6 2015, 01:38:48) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> print('Hello World!')
Hello World!
>>>
===== RESTART: C:\Users\dlmao\PythonClass\helloworld.py =====
Hello World!
Press any key...
>>>
```

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-

def main():
    print('Hello World!')
    input('Press any key...')

# pass

if __name__ == '__main__':
    main()
```

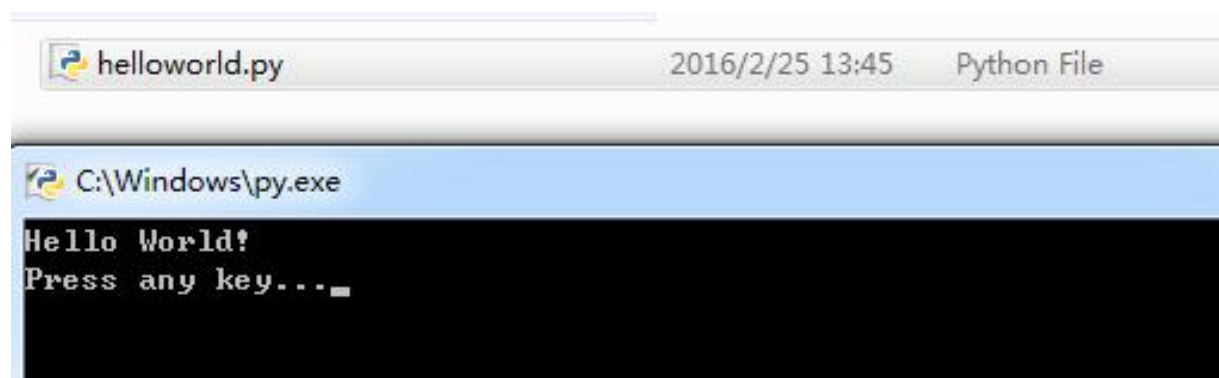
IDLE快捷键

- 撤销 (Ctrl+Z)、全选 (Ctrl+A)、复制 (Ctrl+C)、粘贴 (Ctrl+V)、剪切 (Ctrl+X)

快捷键	功能说明
Alt+p	浏览历史命令（上一条）
Alt+n	浏览历史命令（下一条）
Ctrl+F6	重启Shell，之前定义的对象和导入的模块全部失效
F1	打开Python帮助文档
Alt+/ 	自动补全前面曾经出现过的单词，如果之前有多个单词具有相同前缀，则在多个单词中循环选择
Ctrl+]	缩进代码块
Ctrl+[	取消代码块缩进
Alt+3	注释代码块
Alt+4	取消代码块注释。

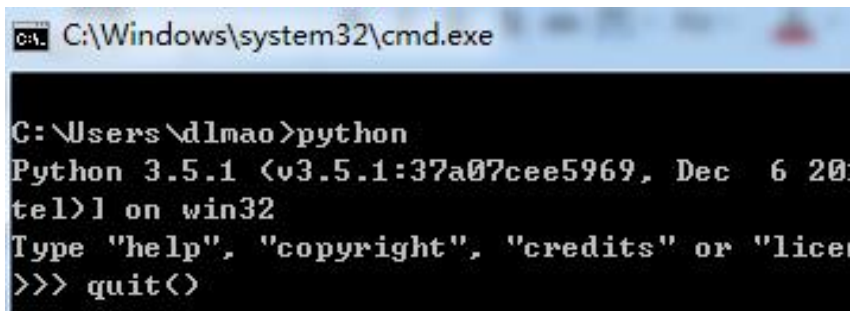
Python程序运行:双击Python程序

- 资源管理器中双击扩展名为“.py”或“.pyc”的Python程序文件直接运行



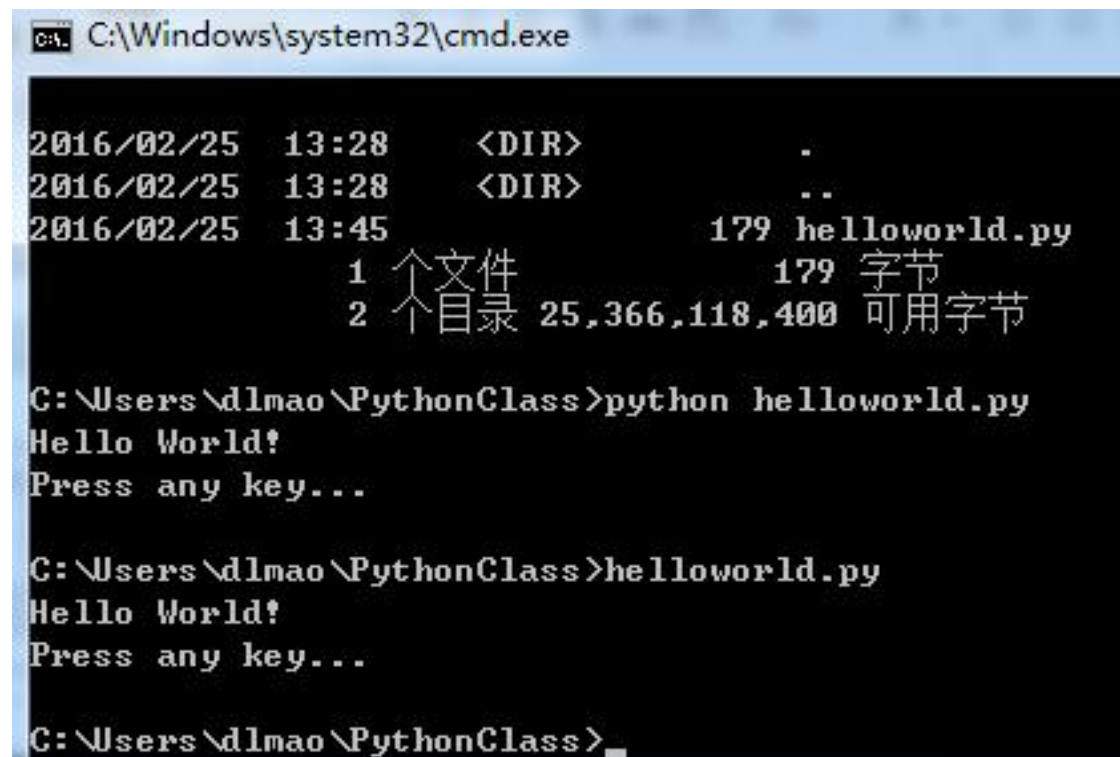
Python程序运行:命令行

- 命令提示符环境中运行Python程序文件。在“开始”菜单的“附件”中单击“命令提示符”，然后执行Python程序
 - python
 - python helloworld.py
 - helloworld.py



```
C:\Windows\system32\cmd.exe

C:\Users\dlmao>python
Python 3.5.1 (v3.5.1:37a07cee5969, Dec  6 2015)
Type "help", "copyright", "credits" or "license()"
>>> quit()
```



```
C:\Windows\system32\cmd.exe

2016/02/25  13:28    <DIR>          .
2016/02/25  13:28    <DIR>          ..
2016/02/25  13:45                179 helloworld.py
                1  个文件                179  字节
                2  个目录 25,366,118,400 可用字节

C:\Users\dlmao\PythonClass>python helloworld.py
Hello World!
Press any key...

C:\Users\dlmao\PythonClass>helloworld.py
Hello World!
Press any key...

C:\Users\dlmao\PythonClass>
```

Python相关的环境变量设置

- 环境变量Path是否包含Python所安装的目录
 - 开始→右击计算机→属性 或者 开始→控制面板→系统和安全→查看计算机的名称
 - 高级系统设置→高级选项卡中的环境变量→用户变量或者系统变量（所有用户都适用）中的Path变量



1.4 Python基础知识: 对象 (Object)

- 对象: Python中各种数据的抽象
 - 标识id(identity): 对象一旦创建其ID不再改变, 可以看成该对象在内存中的地址,
 - id(x) 返回对象x的ID, a is b 判别 a和b是否同一个对象 (ID是否相同)
 - 类型(type): 决定了对象可能取值范围以及支持的操作。强类型语言
 - type(x) 返回对象x的类型
 - 对象是类型的一个实例 (instance)
 - 值(value): 值可能改变或者不可变, 决定于其类型
 - 不可变(immutable): 有些对象一旦创建其值不可变, 比如数字、字符串、元组等
 - 可变(mutable): 对象的值可以改变, 比如列表、字典等

Python 内置对象类型

对象类型	示例
数字	1234, 3.14, 3+4j
字符串	'swfu', "I'm student", '''Python '''
列表	[1, 2, 3] ['a','b',['c',2]]
字典	{1:'food' ,2:'taste', 3:'import'}
元组	(2, -5, 6)
文件	f=open('data.dat', 'r')
集合	set('abc'), {'a', 'b', 'c'}
布尔型	True, False
空类型	None
编程单元类型	函数(def)、模块、类(class)

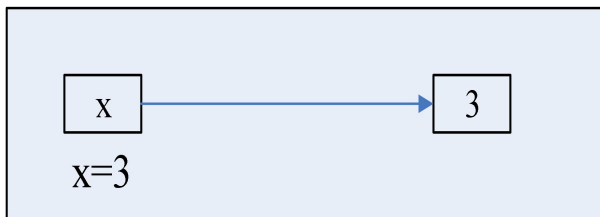
如何描述和引用对象?

- Literal(字面值)
 - 表示某个内置对象类型的固定不变的值, 在词法和语法分析时识别其类型
 - 字符串字面值: 'Hello World'
 - 整数字面值: 2016 00177 0xda80 0b10010111
 - 浮点数字面值: 3.14 10. .001 3.14e-10 1.0e100
 - 复数字面值: 3.14j 10j
- 表达式(expression): 各个对象通过运算符(operator)运算之后的结果
 - $(3*4+5)*6$
- 变量(variable): 表示对于某个对象的引用 (reference)
 - 在程序中通过变量名来描述
 - 在使用之前无需申明变量及其类型, 而是自动判断其类型, 支持的运算由类型决定
 - 通过赋值语句来给变量赋值: LHS = RHS
 - 变量出现在RHS(Right Hand Side)处时表示引用该变量所指向对象的值
 - 变量在使用前必须有定义 (赋值)
 - 变量出现在LHS, 表示给该变量赋值, 即保存RHS所对应对象的引用 (地址)
 - $a = (3*4+5)*6$
 - $a = a*a$

变量

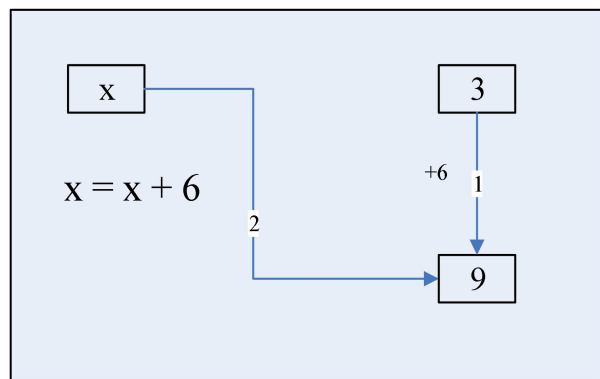
- 由于变量表示对于某个对象的引用，因此Python允许多个变量指向同一个对象

```
>>> x = 3
>>> id(x)
1786684560
>>> y = x
>>> id(y)
1786684560
```



- 一个变量赋予一个新的值后，变量指向新的对象

```
>>> x += 6
>>> id(x)
1786684752
>>> y
3
>>> id(y)
1786684560
```



对于不可变对象，一个取值对应着内存中的唯一一个对象，即在内存中仅存放一个值，如果不同变量为相同值，分别保存了对于同一个对象的引用

```
>>> x = 3
>>> id(x)
10417624
>>> y = 3
>>> id(y)
10417624
>>> y = 5
>>> id(y)
```

But it's NOT TRUE! 字符串也许也是这样，整数仅-5~256才是这样，但同为不可变的元组却不是这样。
事实是：Python对一些常用数值进行了优化，在内存中唯一，对变量赋其他值，其实是产生新的对象。

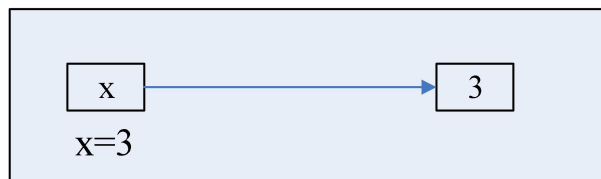
变量

- 由于变量表示对于某个对象的引用，因此Python允许多个变量指向同一个对象

```
>>> x = 3
>>> id(x)
1786684560
```

```
>>> y = x
```

```
>>> id(y)
1786684560
```



- 一个变量赋予一个新的值后，变量指向新的对象

```
>>> x += 6
```

```
>>> id(x)
```

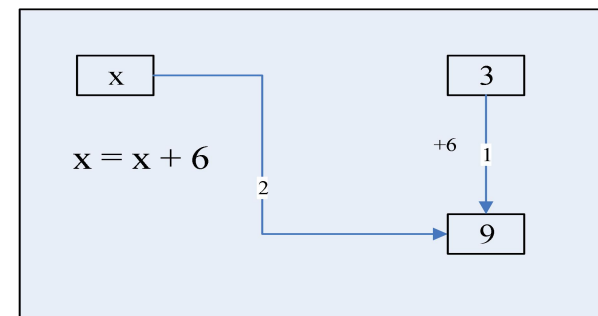
```
1786684752
```

```
>>> y
```

```
3
```

```
>>> id(y)
```

```
1786684560
```



变量

- Python是一种动态类型语言：变量的类型是可以随时变化的，实际上变量保存的是对象的引用

```
>>> x = 3
```

```
>>> print(type(x))
```

```
<class 'int'>
```

```
>>> x = 'Hello world.'
```

```
>>> print(type(x))
```

```
<class 'str'>
```

```
>>> x = [1,2,3]
```

```
>>> print(type(x))
```

```
<class 'list'>
```

```
>>> isinstance(3, int)
```

```
True
```

```
>>> isinstance('Hello world', str)
```

```
True
```

- 内置函数`type(object)`用来返回`object`的类型
- 内置函数`isinstance(obj,class)`用来测试对象`obj`是否为指定类型`class`的实例
- Python具有自动内存管理功能
 - 跟踪所有的对象，并自动删除不再有变量指向的对象。
 - 可以显式使用`del`语句回收那些不再需要的对象所占用的资源
 - `del x`
- `help(thing)`: 查看对象`thing`相关的帮助
- `dir([object])`: 查看`object`相关的属性列表
- `[n for n in dir([object]) if not n.startswith('_')]` 过滤掉以`_`开头的属性

变量名

- 标识符是变量、函数、类、模块和其他对象的名字
- 必须以字母或下划线开头，其后的字符可以是字母、下划线或数字
 - 以下划线开头的变量在Python中有特殊含义
 - 变量名中不能有空格以及标点符号（括号、引号、逗号、斜线、反斜线、冒号、句号、问号等等）
 - 99var It'OK first-class

- 标识符对英文字母的大小写敏感，例如student和Student是不同的变量。
- 一些特殊的标识符保留为Python关键字，不能用作变量名

```
import keyword
print(keyword.kwlist)
keyword.iskeyword (str)  判断是否为关键字
```

```
>>>help()
help>keywords
```

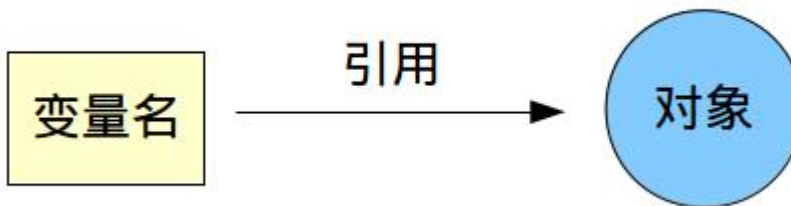
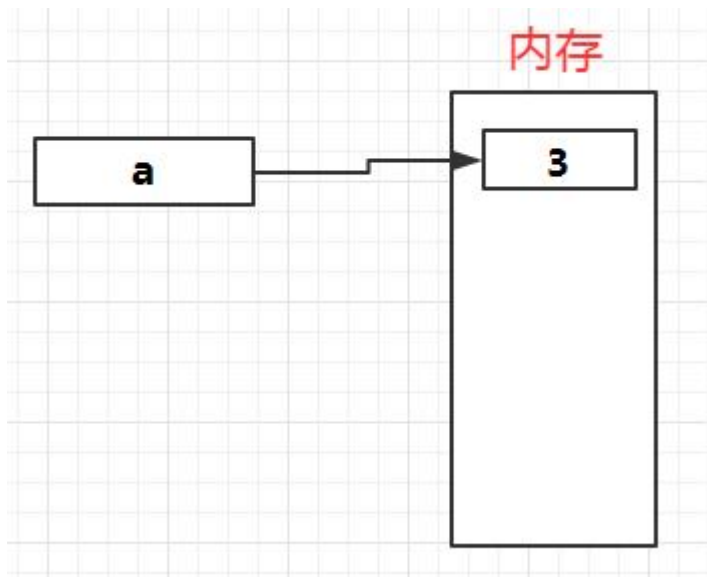
- 不建议使用系统内置的模块名、类型名或函数名以及已导入的模块名及其成员名作变量名，这将会改变其类型和含义，可以通过dir(__builtins__)查看所有内置模块、类型和函数；
- 建议使用有意义的名字，i, j, k仅仅用在较短的循环等结构，比如area、keys等
- 建议使用小写字母表示变量名，多个单词时以下划线隔开：lower_case_with_underscores

变量的进一步说明

- 变量仅仅是对象的引用

例1:

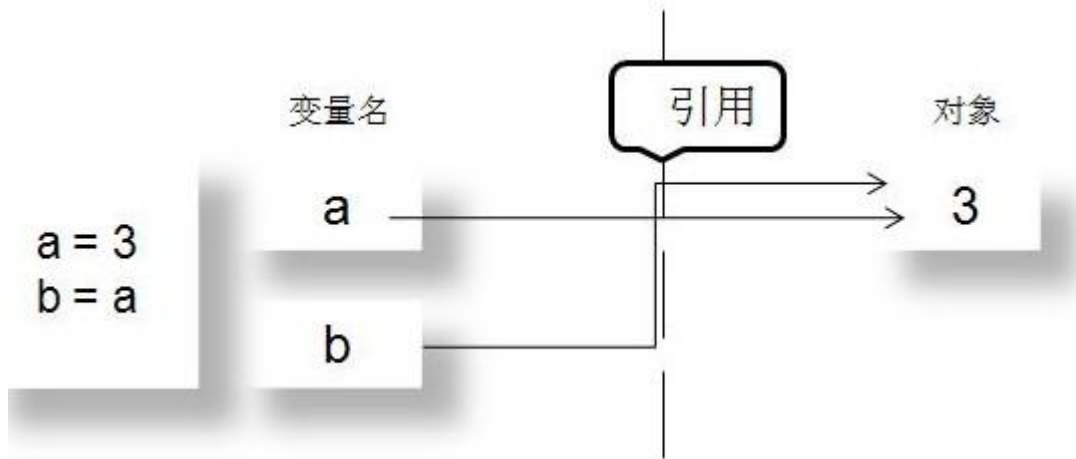
```
>>>a=3
```



例2:

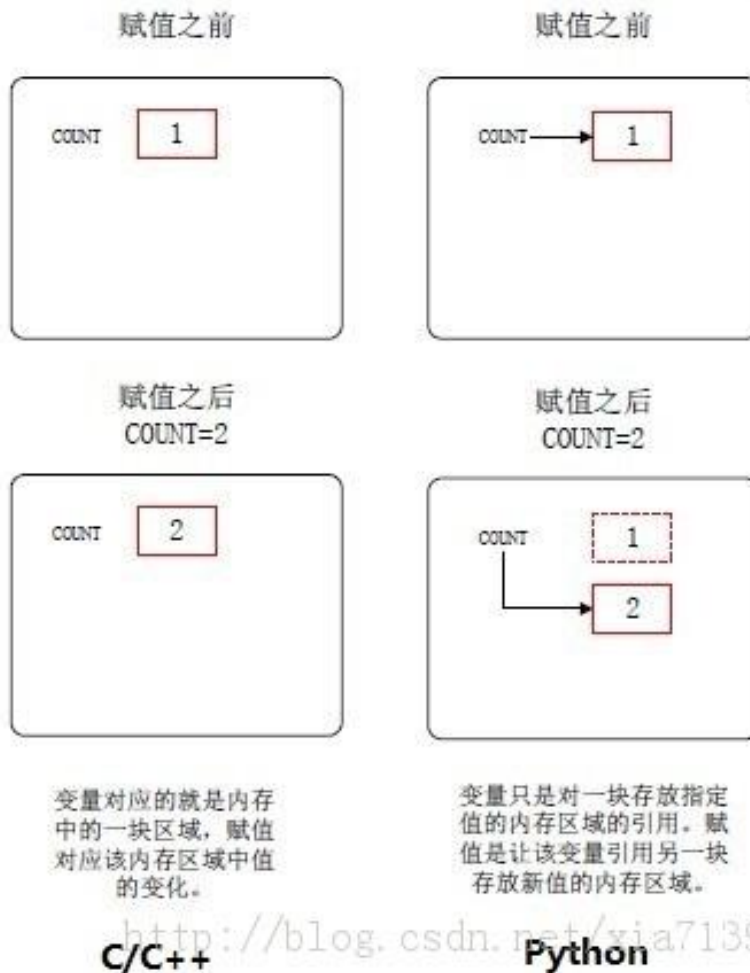
```
>>>a=3
```

```
>>>b=a
```



变量的进一步说明

- 变量赋值与其他语言的区别



1.4.3 数字

- 数字为不可变对象
- 包括整数(int)、浮点数(float)、复数(complex)、布尔 (bool)
- 与有些语言不同，整数可以表示任意大的数值

```
a=99999999999999999999999999999999
```

```
print(a*a*a)
```

- 整数字面值 (integer literal)
 - 十进制整数: 0、-1、9、123
 - 十六进制整数: 0x开头后面为十六进制数字, 即为0-9以及[a-f] (a、b、c、d、e、f) 如0x10、0xfa、0xabcdef, 注意这些数字的大写字母也可以
 - 八进制整数: 以0o开始后面为八进制数字即0-7, 比如0o35、0o11
 - 二进制整数: 以0b开头如, 0b101、0b100、0b10010111

(有限精度) 浮点数 (float)

- 浮点数字面值：浮点数用于表示实数，在内部采用科学计数法表示，因此称为浮点数
 - 小数表示：3.14 10. .001
 - 科学计数法：15e-2 (=15*10⁻² = 0.15) 3.14e-10 1.0e100
 - 需要说明的是由于计算机内部采用二进制表示，与其他语言一样采用IEEE 574双精度表示浮点数时，有精度误差
a= 0.1 + 0.1 + 0.1
a == 0.3 结果为 False
- Python提供了decimal模块（高精度浮点数Decimal）用于十进制数学计算，来保证用户指定的精度
- fractions模块用于表示分数

```
>>>from decimal import Decimal
```

```
>>>Decimal(0.1)
```

```
Decimal('0.1000000000000000055511151231257827021181583404541015625')
```

```
>>>from fractions import Fraction
```

```
>>>Fraction('3/7')
```

```
Fraction(3, 7)
```

复数 (Complex)

- 一个复数包括实部 (real) 和虚部 (imaginary) , 即 实部+虚部
- 虚数字面量:
 - 浮点数或者整数, 后面为j (或者J)
 - 3+4j 1.1+2.2j 3+4.2j 3-4.2j 2.5j+3 4j 0j
 - 注意 j 4*j 都不是复数
- 复数也支持常用的数学运算, 包括+ - * / **等

```
>>> a = 3+4j
>>> b = 5+6j
>>> c = a+b
>>> c
(8+10j)
>>> a*b #复数乘法
(-9+38j)
>>> a/b #复数除法
(0.6393442622950819+0.03278688524590165j)
```

- 复数对象还包括两个属性和方法

```
>>> c.real #查看复数实部
8.0
>>> c.imag #查看复数虚部
10.0
>>> a.conjugate() #返回共轭复数
(3-4j)
```

算术运算符

- + - 加法、减法： $35+4$ $2.5 + 3.5$
- 乘法*： $(35+4)*5$, $3.4*5.2 + 4$ 注意*不能省略
- 除法：
 - 整除// ： $5//2 = 2$ 因为 $5=2*2+1$ $-5//2 = -3$ 因为 $-5 = 2*(-3)+1$
 - 浮点除法/ （也称“真除法”），结果为浮点数： $5/2 = 2.5$ $10/3.2 = 3.125$
- 幂运算**： $4**2 = 16$ $4**0.5 = 2.0$
- 取余（求模）%： $10\%5 = 0$ $20\%3 = 2$ $-20\%3 = 1$
- 一元的改符号+、- ： $+8$ -8

算术运算符

- 优先级顺序，下面四组从低到高，同级的左结合运算： $3-4**2+10\%5+8*6//7$
 - +- 最低
 - * / // %
 - 一元的+-
 - **
- 通过小括号改变运算顺序，可嵌套，**注意：中括号[]和大括号{}不用于数学计算**
- 上述运算符支持两个不同的数字类型的对象之间的运算，即混合运算，比如浮点数+整数
 - 如果其中包含complex对象，则其他数字类型转换为complex对象，结果为complex对象
 - 如果其中包含float对象，则其他数字类型转换为float对象，结果为float对象

比较运算符

- 比较数字对象的值的大小，结果为bool值True或者False

- $x < y$; $x \leq y$; $x > y$; $x \geq y$

- 可以连用，连用时不考虑传递性，仅仅相邻数之间的比较

- $x < y < z$ 表示 $x < y$ 且 $y < z$

- $x < y > z$ 表示 $x < y$ 且 $y > z$

- ```
>>> x,y,z=12,18,13
```

- ```
>>> x < y, x <= y, x > y, x >= y    # (True, True, False, False)
```

- ```
>>> x < y < z, x < y > z # (False, True)
```

- 比较对象的值是否相等

- $x == y$ 

```
>>> x == y # False
```

- $x != y$ 

```
>>> x != y # True
```

# 内置数学函数

| 函数                      | 含义                           | 实例                                 | 结果                      |
|-------------------------|------------------------------|------------------------------------|-------------------------|
| abs(x)                  | x的绝对值。如果x为复数，则返回其模           | abs(-1.2)<br>abs(1-2j)             | 1.2<br>2.23606797749979 |
| divmod(a,b)             | 返回a除以b的商和余数                  | divmod(5,3)                        | (1,2)                   |
| pow(x,y[,z])            | 返回x**y。如果z有，则为pow(x,y)%z     | pow(2,10)<br>pow(2,10,10)          | 1024<br>4               |
| round(number[,ndigits]) | 四舍五入取整，如果ndigits则保留ndigits小数 | round(3.14159)<br>round(3.14159,4) | 3<br>3.1416             |
| max(arg1,arg2,*args)    | 取最大值                         | max(1,7,3,15,14)                   | 15                      |
| min(arg1,arg2,*args)    | 取最小值                         | min(1,7,3,15,14)                   | 1                       |
| sum(iterable[,start])   | 求和，start如果有表示加上start         | sum((1,2,3))<br>sum((1,2,3),44)    | 6<br>50                 |

# math模块

- `import math` 或者 `from math import *`
- `math.pi`: 数学常量  $\pi$     `print(math.pi)` = 3.141592653589793
- `math.e`: 数学常量  $e$     `math.e` = 2.718281828459045

| 函数                         | 含义                            | 实例                                    | 结果                 |
|----------------------------|-------------------------------|---------------------------------------|--------------------|
| <code>fabs(x)</code>       | 绝对值, 返回float                  | <code>fabs(-3)</code>                 | 3.0                |
| <code>ceil(x)</code>       | 大于等于x的最小的整数                   | <code>ceil(1.2),ceil(-1.6)</code>     | (2,-1)             |
| <code>floor(x)</code>      | 小于等于x的最大的整数                   | <code>floor(1.8),floor(-2.1)</code>   | (1,-3)             |
| <code>trunc(x)</code>      | 截取为最接近0的整数                    | <code>trunc(1.2),trunc(-2.8)</code>   | (1,-2)             |
| <code>factorial(x)</code>  | 整数 $x(\geq 0)$ 的阶乘            | <code>factorial(5)</code>             | 120                |
| <code>sqrt(x)</code>       | $x(\geq 0)$ 的平方根              | <code>sqrt(2)</code>                  | 1.4142135623730951 |
| <code>exp(x)</code>        | $e^{**x}$                     | <code>exp(5)</code>                   | 148.4131591025766  |
| <code>log(x[,base])</code> | 以base为底的对数, base没有则为以e为底的自然对数 | <code>log(math.e**2), log(4,2)</code> | (2.0,2.0)          |

# math模块:三角函数和角度转换

| 函数                      | 含义        | 实例                       | 结果                 |
|-------------------------|-----------|--------------------------|--------------------|
| <code>sin(x)</code>     | x的正弦      | <code>sin(pi/2)</code>   | 1.0                |
| <code>cos(x)</code>     | x的余弦      | <code>cos(2*pi)</code>   | 1.0                |
| <code>tan(x)</code>     | x的正切      | <code>tan(pi/4)</code>   | 0.9999999999999999 |
| <code>asin(x)</code>    | x的反正弦     | <code>asin(1)</code>     | 1.5707963267948966 |
| <code>acos(x)</code>    | x的反余弦     | <code>acos(1)</code>     | 0.0                |
| <code>atan(x)</code>    | x的反正切     | <code>atan(1)</code>     | 0.7853981633974483 |
| <code>degrees(x)</code> | x从弧度转换为角度 | <code>degrees(pi)</code> | 180.0              |
| <code>radians(x)</code> | x从角度转换为弧度 | <code>radians(90)</code> | 1.5707963267948966 |

# 布尔 (bool)

- 整数类型的特例，取值为True、False，分别对应整数1和0
- 实际上所有非0的整数转变为bool时都是True
- True 等价于 非0，False 等价于 0
- `int(True)=1, int(False) = 0`
- `bool(4)=True, bool(0) = False`

## 1.4.4 字符串(str)

- 字符串属于不可变序列
- 与其他语言一般用双引号来定义字符串字面值不同，Python可以使用单引号和双引号
  - Python3的字符串默认为Unicode字符串，从而可以包含中文字符串
  - 单引号和双引号定义的字符串不能跨越多行
  - 一般建议采用单引号定义
  - 如果字符串中有双引号，则使用单引号定义，反之亦然
- 三引号'''或'''表示的字符串可以换行，用于长字符串，也用于在程序中表示较长的注释。
- 空串表示为"或 ""

```
>>> a = "first line \nline 2\nline 3\nlast line."
>>> a
'first line \nline 2\nline 3\nlast line.'
>>> print(a)
first line
line 2
line 3
last line.
```

# 字符串转义

- 如果一个字符串内容有单双引号，则需要通过在引号前加上转义字符表示为字符串的一部分，而不是字符串的结束
- 回车换行以及其他控制字符也可以通过转义来描述：

| 特殊字符 | 含义          | 特殊字符 | 含义           |
|------|-------------|------|--------------|
| \'   | 一个单引号       | \"   | 一个双引号        |
| \\   | 一个\         | \t   | 制表符          |
| \n   | 换行符         | \r   | 回车           |
| \ddd | 3位八进制数对应的字符 | \xhh | 2位十六进制数对应的字符 |

- 字符串界定符前面加字母r表示原始字符串，其中的特殊字符不进行转义，但字符串的最后一个字符不能是\。

```
>>> a=r'C:\Program Files (x86)\Python35\Lib'
```

```
>>> a
```

```
'C:\\Program Files (x86)\\Python35\\Lib'
```

# 字符串相关内置函数

| 函数         | 含义                                        | 实例                                                            | 结果                          |
|------------|-------------------------------------------|---------------------------------------------------------------|-----------------------------|
| bin(x)     | 把数字x转换为二进制字符串                             | bin(4)                                                        | '0b100'                     |
| oct(x)     | 把数字x转换为八进制字符串                             | oct(25)                                                       | '0031'                      |
| hex(x)     | 把数字x转换为十六进制字符串                            | hex(75)                                                       | '0x4b'                      |
| str(obj)   | 把对象obj转换为字符串                              | str(16), str(1/2)                                             | ('16', '0.5')               |
| int(x[,d]) | 把数字x截取为整数，或将基数为d的数字型字符串x转换为整数，如果d没有缺省为十进制 | int(3.14), int('314')<br>int('ff',16)                         | (3,314)<br>255              |
| float(x)   | 将对象(数字或字符串)x转换为浮点数                        | float(3),float('3.14')<br>float('12e-2')                      | (3.0,3.14)<br>0.12          |
| len(obj)   | 返回序列类对象obj包含的元素个数                         | len('Hello World'),len('')                                    | (11, 0)                     |
| ord(s)     | 返回长度为1的字符串s中的字符对应的Unicode码（ASCII码是其子集）    | ord('a'),ord('A'),ord('你')<br>ord('o'), ord('\n')             | (97, 65, 20320)<br>(48, 10) |
| chr(x)     | 返回Unicode编码为x的字符串(len=1)                  | chr(65),chr(10),chr(20320)<br>chr(ord('d')-ord('a')+ord('A')) | ('A', '\n', '你')<br>'D'     |

# 字符串支持的运算符 + \* %

- + 字符串合并成新的字符串

```
>>> a='abc' + '123' #生成新对象 'abc123'
```

```
>>> b='\141b\x61' + '123' # 'aba123', ord('a')=97, 八进制为141, 十六进制为61
```

- \* 字符串和整数相乘相当于字符串的内容重复多次, 整数小于等于0时生成空字符串 ''

```
>>> a='*-' * 30 # '*-*'
```

```
>>> a= 4*'NE' # 'NENENENE'
```

- format\_string % obj 把对象obj按格式要求format\_string转换为字符串。

| 格式字符                  | 含义                                                     | 示例                                                      | 结果                                |
|-----------------------|--------------------------------------------------------|---------------------------------------------------------|-----------------------------------|
| %s                    | 输出字符串                                                  | 'Hello %s' % 'Mike'                                     | 'Hello Mike'                      |
| %d                    | 输出整数                                                   | 'Length:%d' % 45                                        | 'Length:45'                       |
| %c                    | 输出字符 chr(num)                                          | '%c' % 65                                               | 'A'                               |
| %[width][.precision]f | 输出浮点数, 如果width和precision有表示字符串长度为width, 小数点后precision位 | '%f' % math.pi<br>'%.2f' % math.pi<br>'%7.4f' % math.pi | '3.141593'<br>'3.14'<br>' 3.1416' |

# 字符串格式化

- `format_string % obj` 把对象obj按格式要求format\_string转换为字符串。

```
>>>a = 3.6674
```

```
>>>'%7.3f' % a
```

```
' 3.667'
```

```
>>> "%d:%c"%(65, 65)
```

```
'65:A'
```

```
>>> """My name is %s, and my age is %d"""%('Dong Fuguo', 38)
```

```
'My name is Dong Fuguo, and my age is 38'
```

## 1.4.5 运算符和表达式

| 运算符示例                                         | 功能说明                    |
|-----------------------------------------------|-------------------------|
| <code>x+y</code>                              | 算术加法，列表、元组、字符串合并        |
| <code>x-y</code>                              | 算术减法，集合差集               |
| <code>x*y</code>                              | 乘法，序列重复                 |
| <code>x/y</code>                              | 除法（在Python 3. x中叫做真除法）  |
| <code>x//y</code>                             | 求整商                     |
| <code>-x</code>                               | 相反数                     |
| <code>x%y</code>                              | 余数（对实数也可以进行余数运算），字符串格式化 |
| <code>x**y</code>                             | 幂运算                     |
| <code>x&lt;y; x&lt;=y; x&gt;y; x&gt;=y</code> | 大小比较（可以连用），集合的包含关系比较    |
| <code>x==y; x!=y</code>                       | 相等（值）比较，不等（值）比较         |
| <code>x or y</code>                           | 逻辑或（只有x为假才会计算y）         |
| <code>x and y</code>                          | 逻辑与（只有x为真才会计算y）         |
| <code>not x</code>                            | 逻辑非                     |
| <code>x in y; x not in y</code>               | 成员测试运算符                 |
| <code>x is y; x is not y</code>               | 对象实体同一性测试（地址）           |
| <code> 、^、&amp;、&lt;&lt;、&gt;&gt;、~</code>    | 位运算符                    |
| <code>&amp;、 、^</code>                        | 集合交集、并集、对称差集            |

## 1.4.6 常用内置函数

- 内置函数：是指不需要导入任何模块就可直接使用的函数。
- 执行下列命令可以列出所有内置函数和内置对象
  - `>>>dir(__builtins__)`
- 使用建议：优先考虑使用内置函数，因为内置函数成熟、稳定，且执行速度相对较快。
- 对于初学者来说，`dir()`和`help()`两个内置函数很有用。
  - 例如：  
`>>>import math`  
`>>>dir(math)`  
`>>>help(math.sqrt)`

## 1.4.6 常用内置函数

- `max(iterable)`、`min(iterable)`、`sum(iterable)` 计算可迭代对象中所有元素最大值、最小值以及所有元素之和，
  - `sum` 只支持数值型元素的可迭代对象
  - `max` 和 `min` 则要求元素之间可比较大小。

```
>>> import random
>>> a = [random.randint(1,100) for i in range(10)]
>>> a
[72, 26, 80, 65, 34, 86, 19, 74, 52, 40]
>>> print(max(a), min(a), sum(a), sum(a)/len(a))
86 19 548 54.8
```

# random模块

```
import random
```

```
r=random.randint(0,25)
```

```
print("The character is",chr(r+ord('a'))) # 随机小写英文字母
```

```
import string
```

```
print("The character is",random.choice(string.ascii_lowercase))
```

```
print(random.sample(range(1,100),4)) #1到100之间随机4个不同的整数
```

| 函数            | 含义                 |
|---------------|--------------------|
| random()      | 返回在[0,1)区间的随机实数    |
| uniform(a,b)  | 返回在[a,b]区间的随机实数    |
| randint(a,b)  | 返回在[a,b]区间的随机整数    |
| choice(seq)   | 返回序列seq中的随机一个元素    |
| sample(seq,n) | 从序列seq中随机选择n个不同的元素 |

## 1.4.7 对象的删除

- Python具有自动内存管理功能
  - 解释器会跟踪所有的对象的引用情况，一旦发现某个对象不再有任何变量引用，垃圾回收机制在适当的时候会回收该对象，释放内存资源
  - del语句可显式解除变量与所指向对象之间的绑定
  - del语句可解除列表list等可变序列中的元素的绑定，即删除该元素。

```
>>> x = [1, 2, 3, 4, 5, 6]
>>> y = 3
>>> z = y
>>> print(y)
3
>>> del y #删除对象
>>> print(y)
Traceback (most recent call last):
 File "<pyshell#52>", line 1, in <module>
 print(y)
NameError: name 'y' is not defined
>>> print(z)
3
```

```
>>> del z
>>> print(z)
Traceback (most recent call last):
 File "<pyshell#56>", line 1, in <module>
 print(z)
NameError: name 'z' is not defined
>>> del x[1] #删除列表中指定元素
>>> print(x)
[1, 3, 4, 5, 6]
>>> del x #删除整个列表
>>> print(x)
Traceback (most recent call last):
 File "<pyshell#60>", line 1, in <module>
 print(x)
NameError: name 'x' is not defined
```

## 1.4.8 基本输入输出

- `input(prompt)`函数：首先输出prompt，等待用户输入，直到用户按回车结束，返回用户输入的字符串（不包括最后的回车）
- `print(value1,value2,...,sep=' ',end='\n',file=sys.stdout)`：将多个值转换为字符串并且输出到相应的文件file，这些值之间以sep分隔，最后以end结束，sep缺省为空格，end缺省为换行，file缺省为标准输出

```
>>>x = input('Please input:')
Please input:3
>>> print(type(x))
<class 'str'>
>>> int(x)
3
x,y,z='Mike',4,3.14
print(x,y,z) # Mike 4 3.14
print(x,y,z,sep=', ') # Mike, 4, 3.14
print(x,y,z,sep=', ',end='****')
Mike, 4, 3.14****

>>> fp = open(r'D:\mytest.txt', 'a+')
>>> print('Hello,world!', file = fp)
>>> fp.close()
```

## 1.4.9 模块(modules)导入与使用

- Python默认安装仅包含部分基本或核心模块，但用户可以安装大量的扩展模块，pip是管理模块的重要工具。Pip的安装可以参考<http://www.tuicool.com/articles/eiM3Er3/>
- Python启动时，仅加载了很少的一部分模块（包括一些内置的模块builtins），在需要时由程序员显式地加载（可能需要先安装）其他模块。
- 减小运行的压力，仅加载真正需要的模块和功能，且具有很强的可扩展性。
- 可以使用`sys.modules.items()`显示所有预加载模块的相关信息。

# 模块的使用

- `import 模块名 [as 别名]`: 访问模块中的对象, 采用模块名.对象或者别名.对象方式

```
>>>import math
```

```
>>>math.sin(0.5) #求0.5的正弦
```

```
>>>import random as rnd
```

```
>>>x=rnd.random() #获得[0,1) 内的随机小数
```

- 可以使用`dir`函数查看任意模块中所有的对象列表, 如果调用不带参数的`dir()`函数, 则返回当前脚本的所有名字列表。
- 可以使用`help`函数查看任意模块或函数的使用帮助。

# 模块的使用

- `from 模块名 import 对象名[ as 别名]`
  - 仅仅从模块中导入特定的对象，访问对象时不再需要包括模块名
  - 可以减少查询次数，提高执行速度

```
>>> from math import sin
```

```
>>> sin(3)
```

```
0.1411200080598672
```

```
>>> from math import sin as f #别名
```

```
>>> f(3)
```

```
0.1411200080598672
```

- `from math import *`
  - 从模块中导入所有的对象
  - 多个模块中有同样的对象名时造成混乱，因此谨慎使用

# 寻找要导入的模块文件

- 导入模块spam时会从sys.path给出的目录列表中查找

```
>>> import sys
```

```
>>> sys.path
```

```
['', 'C:\\Program Files (x86)\\Python35\\python35.zip', 'C:\\Program Files (x86)\\Python35\\DLLs', 'C:\\Program Files (x86)\\Python35\\lib', 'C:\\Program Files (x86)\\Python35', 'C:\\Program Files (x86)\\Python35\\lib\\site-packages']
```

- 可以看到首先从当前目录中查找，最后是安装目录下的site-packages
- 通过append自定义的目录到sys.path可以扩展搜索路径
- 在导入模块时，会优先导入相应的pyc文件，如果相应的pyc文件与py文件时间不相符，则导入py文件并重新编译该模块。

# 1.5 Python代码编写规范

- python程序是依靠代码块的缩进来体现代码之间的逻辑关系的
  - 代码块的开始
    - 类定义、函数定义、选择结构、循环结构等，行尾出现:(后面可以包括空格等)表示后面应该紧跟缩进的代码块
    - 行尾的冒号表示缩进的开始
  - 缩进结束表示代码块的结束
  - 同一个级别的代码块的缩进量必须相同
- 建议以4个空格为基本缩进单位，不建议采用制表符来缩进
- IDLE可以通过下面的方法进行代码块的缩进和反缩进:

Format → Indent Region/Dedent Region

快捷键为 Ctrl + [ 和 Ctrl + ]

# Python代码编写规范

- 必要的空格与空行
  - 运算符两侧建议使用空格分开
  - 逗号建议在其后面添加一个空格
  - 不同功能的代码块之间、不同的函数定义之间建议增加一个空行以提高可读性。
- 如果一行语句太长，可以在行尾加上\来换行分成多行，但是更建议使用括号来包含多行内容。

```
if a > b and a > c and a > 5 and b > 5 and c > 5 \
 and b > c:
 print('blah')
```

```
if (a > b and a > c and a > 5 and b > 5 and c > 5
 and b > c):
 print('blah')
```

# Python代码编写规范

- 一个好的、可读性强的程序一般包含30%以上的注释。
  - 以#开始，表示本行#之后的内容为注释
  - 不属于任何语句的字符串为注释，经常在函数体开始处添加长注释（三引号），作为docstring

```
def isclosed(self):
```

```
 """True if the connection is closed."""
```

```
 return self.fp is None
```

```
>>> print(isclosed.__doc__)
```

```
True if the connection is closed.
```

- 在IDLE开发环境中，可以通过下面的操作快速注释/解除注释大段内容：
  - Format ➔ Comment Out Region/Uncomment Region
  - 快捷键： ALT+3 ALT+4

# Python代码编写规范

书P26:

(3) 每个import只导入一个模块

- 首先导入Python标准库模块，如os、sys、re
- 导入第三方扩展库，如numpy、scipy
- 导入自己定义和开发的本地模块

(6) 适当使用异常处理结构进行容错，后面将详细讲解。

(7) 软件应具有较强的可测试性，测试与开发齐头并进

## 1.6 Python 文件名

- .py: Python源文件, 由Python解释器负责解释执行。
- .pyw: Python源文件, 常用于图形界面程序文件。
- .pyc: Python字节码文件, 可用于隐藏Python源代码和提高运行速度。对于Python模块, 第一次被导入时将被编译成字节码的形式, 并在以后再次导入时优先使用“.pyc”文件
- .pyo: 优化的Python字节码文件, 使用“python -O -m py\_compile file.py”或“python -OO -m py\_compile file.py”进行优化编译。
- .pyd: 一般是由其他语言编写并编译的二进制文件, 常用于实现某些软件工具的Python编程接口插件或Python动态链接库。

## 1.7 `__name__` 属性

- 每个Python程序在运行时都有一个“`__name__`”属性。
  - 如果程序作为模块被导入，则其“`__name__`”属性的值被自动设置为模块名
  - 如果程序独立运行，则其“`__name__`”属性值被自动设置为“`__main__`”。

例如，假设文件`nametest.py`中只包含下面一行代码：

```
print(__name__)
```

- 直接运行时，结果为：

```
__main__
```

- 而将该文件作为模块导入时得到如下执行结果：

```
>>> import nametest
nametest
```

模块最后一句话通常是

```
if __name__ == '__main__':
 function() # 希望自动运行的函数
```

## 1.8 编写自己的包 (package)

- 有大量模块时可以利用package来组织命名空间和类
- 在包的每个目录中都必须包含一个\_\_init\_\_.py文件，用于表示该目录是一个包。
  - 该文件甚至可以是一个空文件
  - \_\_init\_\_.py文件的主要用途是设置\_\_all\_\_变量以及所包含的包初始化所需的代码。其中\_\_all\_\_变量中定义的对象可以在使用from ...import \*时全部正确导入。

```
import xml.sax
```

```
sax_reader = xml.sax.make_parser()
```

## 1.9 Python编程快速入门

- 问题1: 用户输入一个三位自然数, 计算并输出其百位、十位和个位上的数字。

```
x = int(input('请输入一个三位数: '))
```

```
a = x//100
```

```
b = x//10%10
```

```
c = x%10
```

```
print(a, b, c)
```

## 1.9 Python快速入门

- 问题2：已知三角形的两边长及其夹角，求第三边长。

```
import math
x = input('输入两边长及夹角(度)，以逗号分隔：')
a, b, sita = eval(x) #序列解包
c = math.sqrt(a**2 + b**2 - 2*a*b*math.cos(sita*math.pi/180))
print('c=', c)
```

$$c = \sqrt{a^2 + b^2 - 2ab\cos\theta}$$

## 1.9 Python快速入门

- 问题3: 任意输入三个英文单词, 按字典顺序输出。

```
s = input('x,y,z=')
```

```
x, y, z = s.split(',')
```

```
if x > y:
```

```
 x, y = y, x
```

```
if x > z:
```

```
 x, z = z, x
```

```
if y > z:
```

```
 y, z = z, y
```

```
print(x, y, z)
```

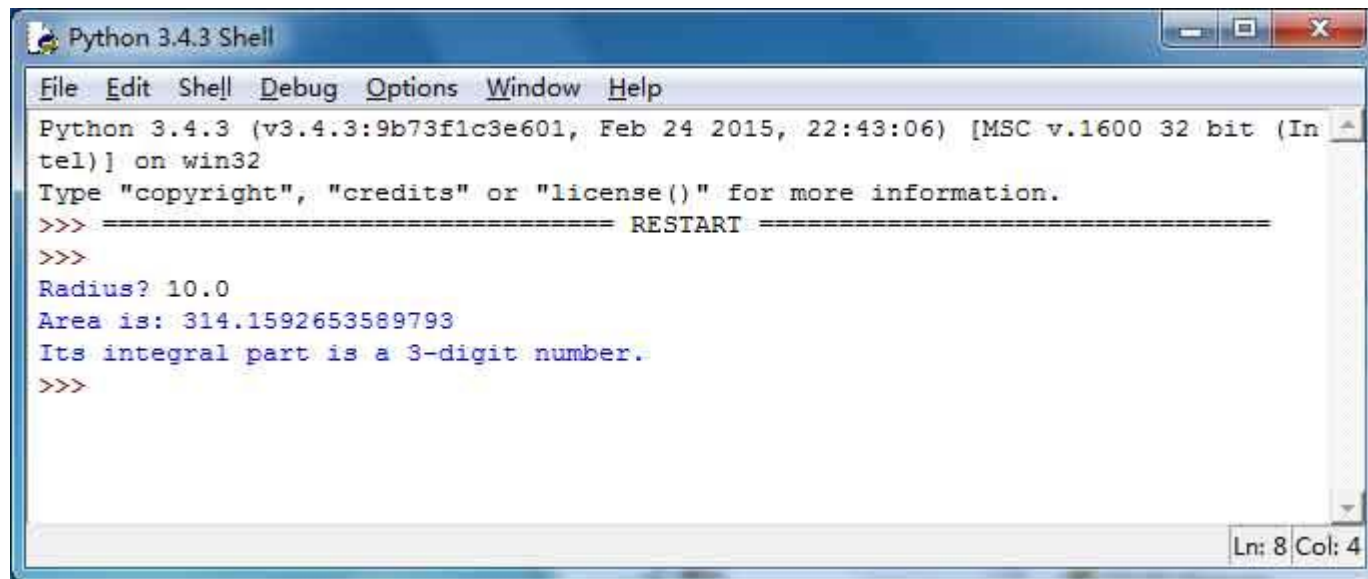
# 1.9 Python快速入门

例4: Python程序框架生成器。

```
import os
import sys
import datetime
head = '# '+'-'*20+'\n'+\
 '# Function description:\n'+\
 '# '+'-'*20+'\n'+\
 '# Author: Dong Fuguo\n'+\
 '# QQ: 306467355\n'+\
 '# Email: dongfuguo2005@126.com\n'+\
 '# '+'-'*20+'\n'
desFile = sys.argv[1]
if os.path.exists(desFile) or not
desFile.endswith('.py'):
 print('%s already exist or is not a Python code
file.!'%desFile)
 sys.exit()
```

```
fp = open(desFile, 'w')
today = str(datetime.date.today().year)+'-'
+str(datetime.date.today().month)+'\
 '-' +str(datetime.date.today().day)
fp.write('# -*- coding:utf-8 -*-\n')
fp.write('# Filename: '+desFile+'\n')
fp.write(head)
fp.write('# Date: '+today+'\n')
fp.write('# '+'-'*20+'\n')
fp.close()
```

# 作业



```
Python 3.4.3 Shell
File Edit Shell Debug Options Window Help
Python 3.4.3 (v3.4.3:9b73f1c3e601, Feb 24 2015, 22:43:06) [MSC v.1600 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> ===== RESTART =====
>>>
Radius? 10.0
Area is: 314.1592653589793
Its integral part is a 3-digit number.
>>>
```

程序有如下功能：

- 1.提示” Radius? “，等待用户输入；
- 2.用户输入一个实数，例如10.0，回车换行；
- 3.程序显示面积；
- 4.程序显示面积的整数部分是几位数。（本步骤选做）

编写代码，保存成.py格式